

Tutorial 7: Advanced Pandas Operations

1. Introduction

In real-world data analysis, datasets rarely come in a clean or single-table format. Data scientists often need to combine datasets, reshape structures, summarize information, and perform complex aggregations.

This tutorial introduces **advanced Pandas operations** used in professional data analysis workflows.

After completing this tutorial, you will be able to:

- Merge datasets from multiple sources
- Reshape data using **melt** and **pivot**
- Create **pivot tables** for summarizing data
- Use **GroupBy operations** for aggregation and analysis
- Perform complex data manipulation efficiently

These techniques are essential for **data analytics, business intelligence, and machine learning workflows**.

2. Merging DataFrames

2.1 Why Merging is Important

Often, data is stored in multiple tables. For example:

- Customer information
- Orders data
- Product details

To perform meaningful analysis, we must **combine these tables using common columns**.

Pandas provides the **merge()** function, which works similarly to **SQL joins**.

2.2 Example Dataset

```
import pandas as pd
customers = pd.DataFrame({
    'customer_id': [1,2,3],
    'name': ['Alice', 'Bob', 'Charlie']
})
orders = pd.DataFrame({
    'customer_id': [1,2,1,3],
    'order_amount': [250,150,300,200]
})
```

2.3 Inner Join

An **inner join** returns only rows that have matching values in both tables.

```
merged = pd.merge(customers, orders, on='customer_id')
print(merged)
```

Output:

customer_id	name	order_amount
1	Alice	250
1	Alice	300
2	Bob	150
3	Charlie	200

2.4 Left Join

A **left join** keeps all rows from the left table and matches rows from the right table.

```
merged = pd.merge(customers, orders, on='customer_id',
how='left')
print(merged)
```

Types of joins available in Pandas:

Join Type	Description
inner	Only matching rows
left	All rows from left table
right	All rows from right table
outer	All rows from both tables

2.5 Merging on Multiple Columns

Sometimes one column is not enough to uniquely identify rows.

```
pd.merge(df1, df2, on=['id', 'date'])
```

This is common in **time-series datasets and transaction data**.

3. Concatenating DataFrames

3.1 Using concat()

When datasets have the **same structure**, they can be stacked together using **concat()**.

```
df1 = pd.DataFrame({'A': [1, 2]})
df2 = pd.DataFrame({'A': [3, 4]})
result = pd.concat([df1, df2])
print(result)
```

Output:

```
A
1
2
3
4
```

Reset index if needed:

```
pd.concat([df1, df2], ignore_index=True)
```

4. Reshaping Data

Reshaping changes the **structure of a dataset**.

Two important reshaping operations are:

- **Melt (Wide → Long format)**
- **Pivot (Long → Wide format)**

4.1 Melt (Wide to Long Format)

Example dataset:

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob'],
    'Math': [85, 90],
    'Science': [88, 92]
})
```

Convert columns into rows:

```
melted = pd.melt(df,
                  id_vars=['Name'],
                  var_name='Subject',
                  value_name='Score')

print(melted)
```

Output:

Name	Subject	Score
Alice	Math	85
Bob	Math	90
Alice	Science	88
Bob	Science	92

This format is often required for **data visualization libraries such as Seaborn or Plotly**.

4.2 Pivot (Long to Wide Format)

Pivot converts rows into columns.

```
pivoted = melted.pivot(index='Name',
                        columns='Subject',
                        values='Score')

print(pivoted)
```

Output:

Subject	Math	Science
Alice	85	88
Bob	90	92

5. Pivot Tables

Pivot tables summarize data using aggregation functions.

Example dataset:

```
data = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR'],
    'Employee': ['A', 'B', 'C', 'D'],
    'Salary': [60000, 65000, 50000, 52000]
})
```

5.1 Creating a Pivot Table

```
table = pd.pivot_table(data,
                        values='Salary',
                        index='Department',
                        aggfunc='mean')

print(table)
```

Output:

Department	
HR	51000
IT	62500

5.2 Multiple Aggregations

```
pd.pivot_table(data,
                values='Salary',
                index='Department',
                aggfunc=['mean', 'max', 'min'])
```

Output:

	mean	max	min
HR	51000	52000	50000
IT	62500	65000	60000

Pivot tables are widely used in **business analytics dashboards and reports**.

6. GroupBy Operations

6.1 What is GroupBy?

GroupBy operations split data into groups and apply operations such as:

- Sum
- Mean
- Count
- Maximum
- Minimum

The process follows the principle:

Split → Apply → Combine

6.2 Basic GroupBy Example

```
data = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR'],
    'Salary': [60000, 65000, 50000, 52000]
})
grouped = data.groupby('Department').mean()
print(grouped)
```

Output:

Department	
HR	51000
IT	62500

6.3 Multiple Aggregations

```
data.groupby('Department').agg(['mean', 'max', 'min'])
```

Output:

	Salary		
	mean	max	min
HR	51000	52000	50000
IT	62500	65000	60000

6.4 GroupBy with Multiple Columns

```
sales = pd.DataFrame({
    'Region': ['East', 'East', 'West', 'West'],
    'Product': ['A', 'B', 'A', 'B'],
    'Revenue': [1000, 1500, 2000, 1800]
})
sales.groupby(['Region', 'Product']).sum()
```

Output:

Region	Product	Revenue
East	A	1000
East	B	1500
West	A	2000
West	B	1800

7. Advanced GroupBy Transformations

Using transform()

The **transform()** function performs calculations while **preserving the original DataFrame shape**.

Example: Normalize salaries within departments.

```
data['dept_mean'] = data.groupby('Department')
['Salary'].transform('mean')
data['normalized_salary'] = data['Salary'] /
data['dept_mean']
```

This is useful for **feature engineering in machine learning**.

8. Practical Data Manipulation Workflow

Example combining multiple techniques:

```
orders = pd.DataFrame({
    'customer': ['A', 'A', 'B', 'B', 'C'],
    'product': ['Laptop', 'Phone', 'Laptop', 'Tablet', 'Phone'],
    'price': [1000, 500, 1000, 700, 500]
})
summary =
orders.groupby(['customer', 'product']).sum().reset_index()
pivot = summary.pivot(index='customer',
                        columns='product',
                        values='price')

print(pivot)
```

Output:

product	Laptop	Phone	Tablet
customer			
A	1000	500	NaN
B	1000	NaN	700
C	NaN	500	NaN

9. Exercises

1. Merge two datasets:

```
import pandas as pd
```

```
students = pd.DataFrame({  
    'id': [1, 2, 3, 4, 5],  
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma']  
})
```

```
grades = pd.DataFrame({  
    'id': [1, 2, 3, 5, 6],  
    'score': [85, 90, 78, 88, 92]  
})
```

2. Convert a wide dataset into long format using **melt()**.
3. Create a **pivot table** showing average salary by department.
4. Use **groupby()** to calculate total revenue per region.
5. Normalize sales values within each product category using **transform()**.